PRODUCT REQUIREMENTS DOCUMENT

Tether

A Source-Mapped Transpiler for Debuggable High-Level Code

Field	Value
Document	Product Requirements Document (PRD)
Project	Tether — a source-mapped transpiler
Author	Antra
Registration Number	24BDS0066
Subject	Compiler Design
Version	1.0
Date	25 September 2026
Status	Draft for review

1. Overview

Tether compiles a small, statically typed imperative language to ECMAScript and emits a conformant Source Map v3 file beside the generated output. The map lets an ordinary JavaScript debugger — one that knows nothing about the source language — report errors, stack traces and breakpoints against the code the programmer actually wrote.

The engineering premise of the project is that source position is not a diagnostic convenience to be discarded after the error message is printed, but a first-class artifact that must survive lexical analysis, parsing, semantic analysis, lowering and code generation intact. Every requirement below follows from that premise.

2. Goals and Non-Goals

Goals

- G1 — Compile a frozen, documented subset of the source language to JavaScript that runs unmodified on Node 18+ and in current browsers.
- G2 — Carry a (line, column, length) span from token through AST, lowering and emission without loss.
- G3 — Emit a Source Map v3 file that an off-the-shelf debugger accepts and resolves correctly.
- G4 — Report several source-level diagnostics per run, each anchored to a precise span.
- G5 — Measure how much positional fidelity is lost when optimisation is enabled.

Non-Goals

- N1 — Competitive runtime performance of the generated JavaScript. Correctness and traceability outrank speed.
- N2 — A general-purpose language. The subset exists to exercise compiler phases, not to be adopted.
- N3 — A second back end. One target keeps the source-map work central.
- N4 — Editor tooling, language server, or incremental compilation.

3. Users and Use Cases

User	Need	How Tether serves it
Program author	Write a program, run it, and debug it when it misbehaves	Breakpoints, stack traces and variable names appear in terms of the original source, not the generated JavaScript
Course evaluator	See each compiler phase demonstrated and inspect its output	Dump flags expose the token stream, AST with spans, lowered core and mapping table
Compiler developer	Change one phase without breaking the rest	Phases are separately unit-testable; golden-file tests cover the pipeline end to end

4. Language Scope

The grammar is frozen at the start of implementation and recorded in BNF as Appendix A of the final report. A change to the grammar requires a revision of this document.

Category	In scope
Types	int, bool, string
Declarations	let bindings with inferred or annotated type; function declarations
Expressions	literals, identifiers, unary − and !, binary arithmetic and comparison, && and ||, calls, parentheses
Statements	expression statement, assignment, if / else, while, for, return, block, print
Functions	named, typed parameters, single return type, recursion permitted

5. Functional Requirements

Priorities follow MoSCoW. Must-priority requirements define a passing implementation; Could-priority requirements are attempted only after every acceptance criterion in section 8 is met.

5.1 Lexical analysis

ID	Requirement	Priority
FR-L1	The scanner shall emit a token stream in which every token carries its source file, start line, start column and length.	Must
FR-L2	Positions shall be 1-based in every user-facing diagnostic and 0-based in every source-map field. Conversion shall happen at exactly one boundary in the code.	Must
FR-L3	The scanner shall recognise integer, string and boolean literals, identifiers, keywords, operators and line comments.	Must
FR-L4	An unterminated string literal shall be reported against the span of its opening quote.	Must
FR-L5	A tab shall advance the column counter by one, and that convention shall be documented.	Should

5.2 Syntax analysis

ID	Requirement	Priority
FR-P1	The parser shall be hand-written recursive descent, using precedence climbing for binary operators.	Must
FR-P2	Each AST node shall carry a span equal to the union of its children’s spans; a leaf takes its token’s span.	Must
FR-P3	On a syntax error the parser shall recover at the next statement boundary and continue, reporting up to 20 syntax errors in one run.	Must
FR-P4	The flag --dump-ast shall print the tree with each node’s span.	Should

5.3 Semantic analysis

ID	Requirement	Priority
FR-S1	A scoped symbol table shall resolve every identifier to its declaration. Shadowing in a nested block is legal; redeclaration in the same scope is an error.	Must
FR-S2	Static type checking shall reject mismatched operand types, non-boolean conditions, wrong argument count, wrong argument type and a missing return on a value-returning path.	Must
FR-S3	Every diagnostic shall print file:line:column, the offending source line, and a caret run beneath the exact span.	Must
FR-S4	Code generation shall not run if any error-severity diagnostic was raised; the process shall exit 1.	Must

5.4 Lowering

ID	Requirement	Priority
FR-M1	A lowering pass shall desugar for into while, compound assignment into binary operation plus assignment, and && / || into conditional expressions.	Must
FR-M2	Every node introduced by lowering shall inherit the span of the construct that caused it to be created.	Must
FR-M3	The flag --dump-core shall print the lowered tree.	Should

5.5 Code generation

ID	Requirement	Priority
FR-G1	The emitter shall produce ECMAScript 2015 that runs unmodified under Node 18+ and in current Chrome and Firefox.	Must
FR-G2	The emitter shall own the only write path to the output buffer and shall update output line and column counters on every write.	Must
FR-G3	An identifier colliding with a JavaScript reserved word shall be renamed deterministically, and its original name retained for the map.	Must
FR-G4	The final line of the generated file shall be the sourceMappingURL comment naming the map file.	Must

5.6 Source map generation

ID	Requirement	Priority
FR-X1	The compiler shall write a JSON file carrying version 3, file, sources, sourcesContent, names and mappings.	Must
FR-X2	The mappings string shall be Base64 VLQ, with a semicolon per generated line and a comma per segment; segments shall carry five fields where a name applies and four otherwise.	Must
FR-X3	Delta encoding shall be correct: the generated column resets at each generated line, while source index, source line, source column and name index accumulate across the whole file.	Must
FR-X4	A mapping shall be emitted for at least the first token of every statement and for every identifier reference.	Must
FR-X5	sourcesContent shall embed the original source so the debugger needs no access to the source file.	Should

5.7 Command-line interface

ID	Requirement	Priority
FR-C1	The CLI shall accept: tether build <file> [-o <dir>] [-O1] [--no-map].	Must
FR-C2	Exit codes shall be 0 on success, 1 on compile error, 2 on internal error.	Must

ID	Requirement	Priority
FR-C3	The flags --dump-tokens, --dump-ast, --dump-core and --dump-mappings shall expose each phase’s output.	Should

5.8 Optimisation and measurement

ID	Requirement	Priority
FR-O1	Under -O1 the compiler shall perform constant folding and dead-code elimination.	Could
FR-O2	A folded expression shall take the span of the whole expression it replaces.	Could
FR-O3	The flag --map-report shall print the proportion of source statements that retain a mapping, for -O0 and -O1.	Could

6. Non-Functional Requirements

ID	Requirement	Rationale
NFR-1	No parser generator or third-party compiler framework; the lexer and parser are hand-written.	These are the phases the course assesses
NFR-2	Compilation shall be deterministic: identical input yields byte-identical .js and .js.map.	Golden-file testing depends on it
NFR-3	Any benchmark program shall compile in under 200 ms on commodity hardware.	Keeps the edit-compile-debug loop usable
NFR-4	Every emitted map shall be verified against an independent reference decoder in the test suite.	A self-consistent encoder can still be wrong
NFR-5	Generated JavaScript shall require no runtime library.	The output must be inspectable as plain code
NFR-6	Each phase shall be unit-testable in isolation, with end-to-end golden-file tests over the pipeline.	A defect should localise to one phase

7. Source Map Conformance

Conformance is judged against the Source Map Revision 3 specification. The following properties are verified mechanically rather than by inspection.

- The mappings string decodes without error under an independent reference decoder.
- Every decoded segment resolves to a position that exists in the named source file.
- For every entry in the compiler’s own mapping table, querying the decoded map at that generated position returns the same original position.
- Every name index referenced by a segment is within the bounds of the names array.
- The generated file’s sourceMappingURL resolves relative to the file’s own directory.

8. Acceptance Criteria

ID	Criterion	Verified by
AC-1	Each of the four benchmark programs produces exactly its expected output when run under Node.	Golden-file test suite
AC-2	A breakpoint set on a line of original source in Chrome DevTools halts at the corresponding statement, demonstrated for at least ten distinct statements.	Recorded screenshots and a written walkthrough
AC-3	Round-tripping every entry of the compiler’s mapping table through an independent decoder returns the original position in 100% of cases.	Automated test
AC-4	A program containing five distinct semantic errors reports all five in one run, each with a correctly placed caret.	Fixture program and expected-output diff
AC-5	A runtime exception’s stack trace, viewed in the browser, names the original source file and line.	Recorded screenshot
AC-6	Mapping retention figures are recorded for -O0 and -O1 across all four benchmarks.	--map-report output in the final report

9. Milestones

Week	Milestone	Exit criterion
1	Grammar frozen and written in BNF; scanner with spans	--dump-tokens prints a span for every token
2	Recursive-descent parser and AST	--dump-ast prints spans; benchmarks parse
3	Symbol table, type checking, diagnostics with carets	AC-4 passes
4	Lowering pass; naive JavaScript emitter, no map	AC-1 passes
5	Base64 VLQ encoder and mapping table	Encoder passes published test vectors
6	Source map emission and reference-decoder validation	AC-3 passes
7	Browser verification; screenshots; stack-trace check	AC-2 and AC-5 pass
8	Optional -O1 pass and fidelity measurement; final report	AC-6 recorded; report submitted

10. Risks and Mitigations

Risk	Impact	Mitigation
Delta encoding implemented incorrectly: the map loads without error but points to the wrong places	High	Unit-test the VLQ encoder against published vectors in week 5, before any integration; validate with an independent decoder from week 6
Off-by-one between 1-based diagnostics and 0-based map fields	High	A single conversion function at the emission boundary, covered by a property test over random positions
Emitter column counters drift when a construct writes across lines	Medium	The emitter owns the only write path; an assertion compares the counters against the final output string after every compile
Scope creep into closures, arrays or user-defined types	Medium	Grammar frozen in week 1; any change requires revising this document
Optimisation work displaces core requirements	Medium	-O1 is Could-priority and is not started until AC-1 through AC-5 pass

11. Out of Scope

Closures and functions as values; user-defined types, arrays and objects; modules and separate compilation; generics; garbage collection; a second back end; a language server or editor integration; incremental or parallel compilation.

12. Deliverables

- Compiler source with unit and golden-file tests.
- The grammar in BNF.
- Four benchmark programs with their generated .js and .js.map pairs.
- Screenshots evidencing breakpoints and stack traces resolved to original source.
- Mapping-fidelity measurements for -O0 and -O1.
- Final project report and presentation.

13. Open Questions

- Should version 1 map expression-level positions, or statement-level only? Statement-level is cheaper and sufficient for breakpoints; expression-level improves stack traces.
- Is a sourceRoot field needed, or is a single relative source path sufficient for the intended directory layout?
- Should a tab advance the column by one or to the next multiple of four? The choice must match whatever the debugger assumes.